TRUSTRAG

Complete Student Guide & Viva Preparation

Everything you need to understand, explain, and defend

your AIML Lab Research Project

Project Name	TRUSTRAG — Evidence-Grounded Hallucination Detection System
Submitted For	AIML Lab Research Paper
Steps Completed	17 Steps (Ingestion to Streamlit UI)
Test Suite	190 tests passing, 0 failures
Models Used	MiniLM-L6-v2 | ms-marco-MiniLM-L-6-v2 | DeBERTa-v3-small | Gemini
Language	Python 3.12.5
Key Result	CrossEncoder improved MRR by +168.6%; Correction reduced errors to 0

This guide explains what you built, why each part exists, how it works, what models you used, what results you got, and how to answer every possible viva question. Read this fully before your submission/viva.

Chapter 1: What Is This Project? (The Big Picture)

1.1 The Core Problem

Modern AI systems like ChatGPT, Gemini, and others are called **Large Language Models (LLMs)**. They are very good at generating text, but they have a critical flaw: they sometimes **make up facts that are not true**. This is called **hallucination**.

For example, if you ask an AI "Who invented data mining?" it might confidently answer "It was invented at MIT in 1985" — even if no document you gave it says that. The AI is hallucinating — generating a plausible-sounding but unverified answer.

1.2 What is RAG?

RAG stands for Retrieval-Augmented Generation. Instead of letting the AI answer from its training memory alone, RAG first *retrieves relevant passages* from your documents, then passes those passages to the AI to generate an answer. This reduces hallucinations because the AI now has actual evidence to refer to.

Think of it this way: Standard AI is like a student answering an exam from memory. RAG is like a student answering with an open book. But even with the book open, the student might still write something wrong. **TRUSTRAG checks that too.**

1.3 What is TRUSTRAG?

TRUSTRAG is a complete system that does the following in order:

Step 1-4: Take your PDF, extract the text, break it into chunks, convert to numbers (vectors), and store in a searchable database.

Step 5-6: When you ask a question, find the most relevant chunks using FAISS search, then re-order them by exact relevance using a CrossEncoder model.

Step 7-8: Build a structured prompt from the evidence and ask Gemini (Google's AI) to generate an answer.

Step 9: Break the AI's answer into individual factual claims (e.g., "Data preprocessing removes noise", "Data mining has 5 steps").

Step 10-11: For each claim, find specific evidence from the document and use a verification model to decide: SUPPORTED, UNSUPPORTED, or CONTRADICTED.

Step 12: Correct or remove any claims that were not supported/were contradicted, using only the evidence — never making up new facts.

Step 13: Generate a Reliability Report showing every claim, its verification result, and its evidence source.

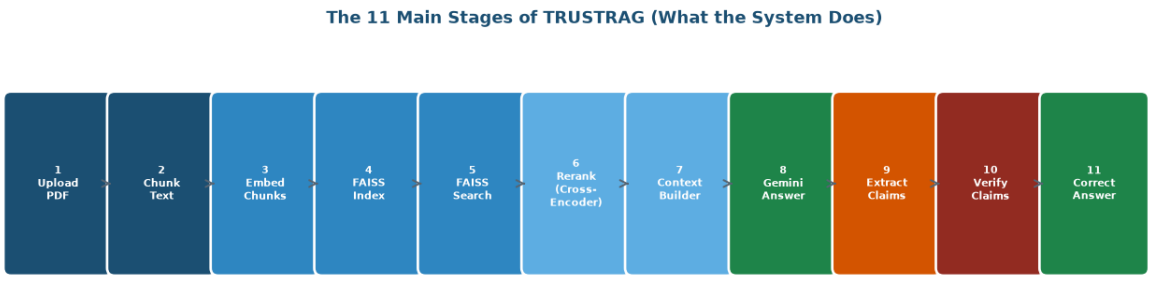


Figure 1: The 11 main processing stages of TRUSTRAG.

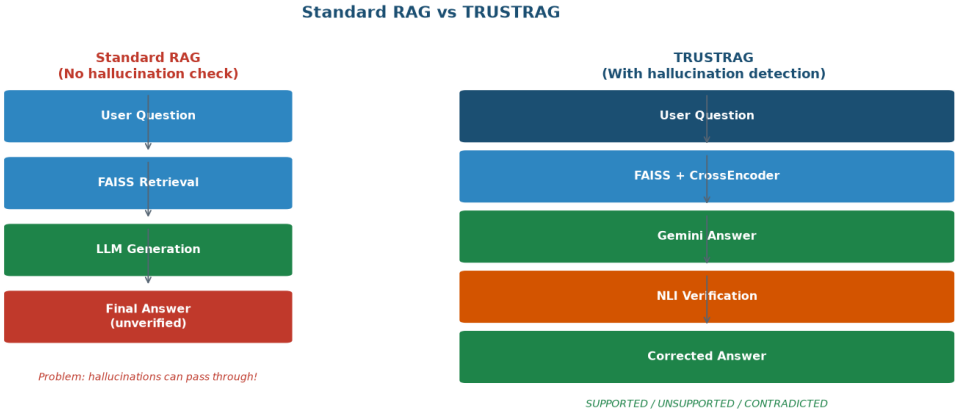


Figure 2: Standard RAG vs TRUSTRAG — the key difference is claim-level verification.

Chapter 2: Terminology — Every Word You Need to Know

LLM (Large Language Model)

A very large AI model trained on billions of text documents. It can generate human-like text. Examples: Gemini (Google), GPT-4 (OpenAI). In our project we used Gemini.

Hallucination

When an AI generates a statement that is factually wrong or not supported by any evidence it was given. The AI 'imagines' facts with complete confidence. This is the problem we solve.

RAG (Retrieval-Augmented Generation)

A technique that retrieves relevant document passages first, then feeds them to the LLM as context before generating an answer. Reduces hallucination compared to pure LLM generation.

Embedding

Converting text into a list of numbers (a vector) that captures the meaning. Semantically similar sentences will have similar number-lists. We use the model 'all-MiniLM-L6-v2' which converts each text chunk into 384 numbers.

FAISS (Facebook AI Similarity Search)

A library that stores millions of embedding vectors and can instantly find the ones most similar to a query embedding. Think of it as a semantic search engine. We use IndexFlatIP (Inner Product) which works with normalized (L2) vectors for cosine similarity.

Cosine Similarity

A way to measure how similar two text embeddings are. Score of 1.0 = identical meaning, 0 = completely unrelated. FAISS uses this to rank chunks by relevance to a question.

CrossEncoder

A more powerful relevance model. Unlike embeddings (which look at query and document separately), a CrossEncoder looks at both the question AND the document passage together at the same time, giving much better relevance scores. It is slower but more accurate.

Re-ranking

After FAISS returns the top-10 candidates, the CrossEncoder re-scores and re-orders them. The result is a more accurate top-5 list. This is the two-stage retrieval approach.

NLI (Natural Language Inference)

A task where a model decides the relationship between two sentences: Entailment (one supports the other), Contradiction (one contradicts the other), or Neutral. We use this to verify claims against evidence.

DeBERTa

A powerful BERT-based model from Microsoft. 'DeBERTa-v3-small' is the NLI verification model we used. It takes a (claim, evidence) pair and scores how well the evidence supports or contradicts the claim.

Claim Extraction

Breaking down an AI-generated answer into individual factual statements. E.g., 'Data preprocessing removes noise and handles missing values' becomes two claims: (1) 'Data preprocessing removes noise'

and (2) 'Data preprocessing handles missing values.'

SUPPORTED

The retrieved evidence clearly confirms the claim. The claim is safe to keep.

UNSUPPORTED

The retrieved evidence does not mention or establish the claim. Important: this does NOT mean the claim is wrong — it just means we cannot confirm it from the document.

CONTRADICTED

The retrieved evidence explicitly conflicts with the claim. E.g., claim says '2 types' but evidence says '4 types'. This is a definitive hallucination.

Chunk

A piece of text cut from the document, usually around 500 words (tokens). We chunk because embedding models have token limits and short chunks search better.

MRR (Mean Reciprocal Rank)

A retrieval quality metric. For each query, calculate 1/rank_of_first_relevant_result, then average across all queries. MRR=1.0 means the best result is always rank 1. Higher is better.

Recall@K

Fraction of queries where the correct evidence appears in the top-K results. Recall@1 means the correct evidence is rank 1. Higher is better.

Macro F1

Average of F1 scores across all classes (SUPPORTED, UNSUPPORTED, CONTRADICTED). Treats each class equally regardless of how many examples it has.

Context Engineering

The process of formatting retrieved evidence into a structured prompt for the LLM. We include evidence IDs, document names, page numbers, and the text — so the LLM knows exactly where each piece of information comes from.

Ablation Study

A research experiment where you remove components one by one to see how much each one contributes. We tested A1 (FAISS only) → A2 (+CrossEncoder) → A3 (+Verification) → A4 (+Correction). This shows which component matters most.

Streamlit

A Python library that lets you create web applications quickly. We built an interactive UI using Streamlit so users can upload PDFs, ask questions, and see the full verification trace through a browser.

PyMuPDF (fitz)

A Python library for reading PDF files. We use it to extract text page by page from uploaded PDFs.

Sentence Transformers

A Python library for using pre-trained embedding and CrossEncoder models. We use it to load all-MiniLM-L6-v2 (embedding) and ms-marco-MiniLM-L-6-v2 (reranker).

Gemini

Google's large language model, accessed via the google-genai Python SDK. We use it for (a) generating answers from evidence and (b) correcting hallucinated claims. It requires an API key (GEMINI_API_KEY environment variable).

Chapter 3: The Models We Used — What, Why, and How

3.1 Model Summary Table

Model Name	What It Does	Why We Chose It	Output
all-MiniLM-L6-v2	Converts text chunks into 384-dimensional embedding vectors	Fast, lightweight, excellent semantic similarity performance. Free to use locally.	384-dim float vector per chunk
ms-marco-MiniLM-L-6-v2	CrossEncoder that scores relevance of a (query, passage) pair	Trained on MS MARCO (Microsoft Machine Reading Comprehension) — best-in-class re-ranking	Single relevance score (float)
cross-encoder/nli-deberta-v3-small	NLI model that classifies claim-evidence pairs as entailment/neutral/contradiction	DeBERTa-v3 is state-of-the-art for NLI. 'small' version balances speed and accuracy.	3 scores: entailment, neutral, contradiction
Gemini (google-genai)	LLM for generating answers from evidence context, and rewriting hallucinated claims	Google's most capable publicly available API. Temperature=0 for deterministic outputs.	Natural language text (answers, corrections)

3.2 How the Embedding Model Works

The model **all-MiniLM-L6-v2** is a Sentence Transformer — it reads a sentence and outputs 384 numbers. These numbers capture the *meaning* of the sentence, not just the words. Two sentences like 'The car is fast' and 'The vehicle moves quickly' will have similar number-lists even though they share no words. This is what makes semantic search possible.

In our system: Every chunk from the PDF is embedded into a 384-dim vector and stored in FAISS. When you ask a question, the question is also embedded into a 384-dim vector, and FAISS finds the chunks whose vectors are most similar to the question vector.

3.3 How the CrossEncoder Re-ranker Works

The embedding model encodes each text independently — it never compares question to passage directly. The CrossEncoder model **ms-marco-MiniLM-L-6-v2** takes both the question AND the passage as a single input and scores their exact relevance together. This is more accurate but slower.

Pipeline: FAISS returns top-10 candidates (fast but imperfect ranking). CrossEncoder re-scores all 10 pairs and returns the top-5 ordered by true relevance. This two-stage approach gives us both speed (FAISS) and accuracy (CrossEncoder).

3.4 How the NLI Verifier Works

For each extracted claim, we have a piece of evidence. The model **cross-encoder/nli-deberta-v3-small** reads both and produces three scores: how much the evidence *entails* the claim, is *neutral* to it, or

contradicts it.

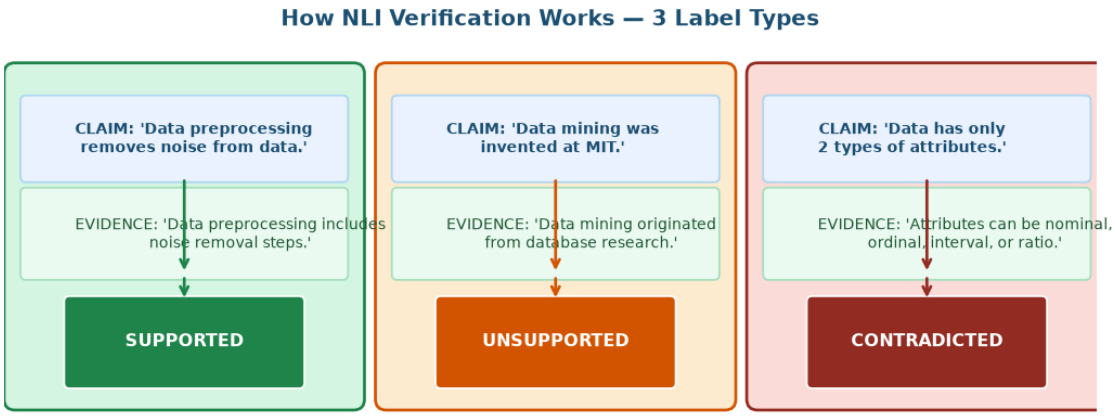


Figure 3: The three possible NLI verification outcomes with examples.

3.5 How Gemini Is Used

Gemini is used at two points: (1) **Answer Generation** — given the structured evidence context and question, generate an initial answer using only the supplied evidence. (2) **Correction** — given the verified claims and evidence, rewrite the answer to remove or fix UNSUPPORTED and CONTRADICTED claims, using only retrieved evidence — never making up facts.

Chapter 4: Step-by-Step — What We Built and How

The project was built in 17 steps over a long development process. Here is what each step did:

Step 1: PDF Ingestion

Module: *ingestion/pdf_loader.py*

We built a module that reads any PDF file using PyMuPDF (fitz). It extracts text page by page, normalises whitespace, and creates PageRecord objects containing the document ID, document name, page number, and text. Empty pages are skipped automatically.

- load_pdf(path) — reads and returns list of PageRecord objects
- generate_document_id() — creates a unique MD5 hash from the file path
- normalize_text() — cleans up multiple spaces/newlines

Step 2: Chunking

Module: *ingestion/chunker.py*

Pages are broken into 'chunks' of approximately 500 words (tokens) with 75-word overlap. Overlap means consecutive chunks share some text, so evidence at chunk boundaries is not lost. If a paragraph is too large, it is split by sentences first, then force-split by words.

- chunk_pages(pages) — returns list of ChunkRecord objects
- TARGET_CHUNK_TOKENS = 500, OVERLAP_TOKENS = 75
- Each chunk has a unique MD5 chunk_id, page_start, page_end, and the text

Step 3: Embeddings

Module: *retrieval/embeddings.py*

Each ChunkRecord's text is embedded using the SentenceTransformer model all-MiniLM-L6-v2. This produces a 384-dimensional float vector per chunk. Vectors are L2-normalised so that cosine similarity = inner product. The model is cached globally after first load.

- embed_chunks(chunks) — returns (np.ndarray of shape [N,384], metadata dict)
- embed_query(text) — returns single 384-dim vector for a question
- Global model cache avoids reloading the model for each query

Step 4: FAISS Index

Module: *retrieval/faiss_store.py*

All chunk embeddings are stored in a FAISS IndexFlatIP index on disk. IndexFlatIP uses exact inner product search (cosine similarity for normalised vectors). The index is saved to disk alongside a JSON metadata file mapping vector positions to chunk_ids.

- build_faiss_index(embeddings) — creates FAISS index from numpy array
- save_faiss_index / load_faiss_index — disk persistence
- search_index(index, query_vector, k) — returns top-k results with scores

Step 5: RAG Retriever

Module: *retrieval/retriever.py*

A reusable class that wraps all retrieval logic. Given a question string, it embeds it, searches FAISS, and returns enriched result dictionaries including text, chunk_id, document name, page range, and FAISS score. Validates all components on startup.

- RAGRetriever(faiss_path, metadata_path, chunks_jsonl_path)
- retrieve(question, k=10) — returns list of result dicts
- Raises descriptive errors if files are missing or dimensions mismatch

Step 6: CrossEncoder Reranker

Module: *retrieval/reranker.py*

Takes the FAISS top-K candidates and re-ranks them using the CrossEncoder model. Each candidate's 'rerank_score' and 'rerank_rank' are added. The original FAISS rank/score are preserved as 'retrieval_rank' and 'retrieval_score' for research traceability.

- CrossEncoderReranker(model_name)
- rerank(question, candidates, top_k=5) — returns re-ranked list
- Lazy model loading — CrossEncoder is only loaded on first call

Step 7: Context Engineering

Module: *context/context_builder.py*

Formats retrieved evidence chunks into a structured prompt for the LLM. Each evidence block includes: Evidence ID (E1, E2...), source document, page numbers, chunk ID, FAISS score, CrossEncoder score, and the actual text. Grounding instructions are appended telling the LLM to use only this evidence.

- ContextBuilder(top_k=5)
- build_context(question, reranked_results) — returns dict with formatted_context, evidence_list
- Validates all required fields in each candidate before formatting

Step 8: LLM Generation

Module: *generation/generator.py + generation/llm_client.py*

LLMGenerator wraps any LLMClient (currently GeminiClient). It sends the formatted context to Gemini at temperature=0 (fully deterministic). The system prompt explicitly instructs Gemini to use only the provided evidence. Retry logic handles 429 (quota) and 503 (unavailable) errors automatically.

- LLMGenerator(client=None) — defaults to get_llm_client()
- generate_answer(context_result) — returns answer string + evidence IDs
- GeminiClient uses GEMINI_API_KEY or LLM_API_KEY environment variable

Step 9: Claim Extraction

Module: *claims/claim_extractor.py*

Uses Gemini to decompose the generated answer into individual atomic factual claims. Only 'factual' claims are extracted — opinions, hedges ('might be'), and meta-statements ('the document says') are excluded. Each claim gets a unique ID (C1, C2, C3...) and a type.

- ClaimExtractor(client=None)
- extract_claims(answer, question) — returns dict with claims list
- Each claim: {claim_id, claim_text, claim_type, confidence}

Step 10: Claim Evidence Retrieval

Module: *claims/evidence_retriever.py*

For each extracted claim, an independent FAISS+CrossEncoder retrieval is run (separate from the original question retrieval). This finds evidence specifically relevant to that individual claim rather than the overall question.

- EvidenceRetriever(retriever, reranker)
- retrieve_for_claims(claims) — returns list of ClaimEvidenceResult
- Each result: {claim_id, claim_text, evidence: [{evidence_id, document, pages, text, scores}]}

Step 11: NLI Verification

Module: *verification/nli_verifier.py*

For each claim, all its evidence pieces are scored using DeBERTa-v3-small NLI. The model outputs entailment/neutral/contradiction scores. We aggregate across evidence pieces: if any evidence strongly entails, SUPPORTED; if any evidence strongly contradicts, CONTRADICTED; otherwise UNSUPPORTED.

- NLIVerifier(model_name='cross-encoder/nli-deberta-v3-small')
- verify(evidence_result) — returns {claim_id, status, score, reason}
- Status ∈ {SUPPORTED, UNSUPPORTED, CONTRADICTED}

Step 12: Evidence-Grounded Correction

Module: *correction/gemini_corrector.py*

Sends verification results to Gemini with a strict prompt: preserve SUPPORTED claims, correct CONTRADICTED claims using the evidence, and remove or qualify UNSUPPORTED claims. The corrector is strictly forbidden from using any knowledge not in the evidence.

- GeminiCorrector(client=None)
- correct(answer, claims, evidence, verifications) — returns {corrected_answer, changes}
- Each change: {claim_id, action: PRESERVED|CORRECTED|REMOVED_OR_QUALIFIED}

Step 13: Reliability Report

Module: *evaluation/reliability_report.py*

Aggregates all pipeline outputs into a structured report. Shows total claims, how many were supported/unsupported/contradicted, claim support rate, problematic claim rate, and for each claim: full evidence chain with document name, pages, chunk ID, and evidence text.

- ReliabilityReportGenerator().generate_report(...) — returns ReliabilityReport
- report.to_dict(), to_json(), to_text() — multiple output formats
- Evidence traceability chain: Claim → Evidence ID → Document → Page → Text

Step 14: Research Report (CLI scripts)

Module: *run_reliability_report.py, run_benchmark.py, etc.*

A set of standalone scripts to run individual pipeline stages from the command line, useful for testing and demonstrating each component independently without running the full UI.

- run_retriever.py, run_reranker.py — test retrieval stages
- run_generation.py — test LLM generation
- run_reliability_report.py — test full pipeline

Step 15: Hallucination Injection Benchmark

Module: *evaluation/benchmark/*

We built a controlled benchmark of 15 test cases by: (a) taking real document content as SUPPORTED claims, (b) creating plausible claims not in the document as UNSUPPORTED, and (c) deliberately introducing factual errors into real claims as CONTRADICTED. This lets us evaluate the NLI verifier with known ground truth.

- BenchmarkDataset — loads the 15 test cases
- HallucinationInjector — modifies real claims to create CONTRADICTED cases
- BenchmarkMetrics — computes accuracy, F1, etc. against ground truth

Step 16: Evaluation & Ablation

Module: *evaluation/experiments/ + run_experiments.py*

Implemented the full ablation study comparing A1 (FAISS only), A2 (+ reranker), A3 (+ verification), A4 (+ correction). Computes Recall@K, MRR, confusion matrix, macro precision/recall/F1, correction success rates, and component latency.

- RetrievalMetricsCalculator — computes MRR, Recall@K
- VerificationMetricsCalculator — confusion matrix, macro metrics
- CorrectionMetricsCalculator — success rates, residual counts
- LatencyMeasurer — per-component timing (local models only)

Step 17: Streamlit UI

Module: *ui/streamlit_app.py*

A full web interface that lets users upload a PDF, ask questions, and see the entire pipeline run with live progress updates. Shows the original answer, reliability summary, per-claim verification panels with evidence chains, the corrected answer, raw FAISS results, latency breakdown, and a research details panel with JSON output.

- streamlit_app.py — main application (8 sections)
- components.py — pure helper functions (fully unit tested)
- session_state.py — manages Streamlit state across reruns
- Gracefully handles Gemini 429 quota errors

Chapter 5: Results — What We Got and What It Means

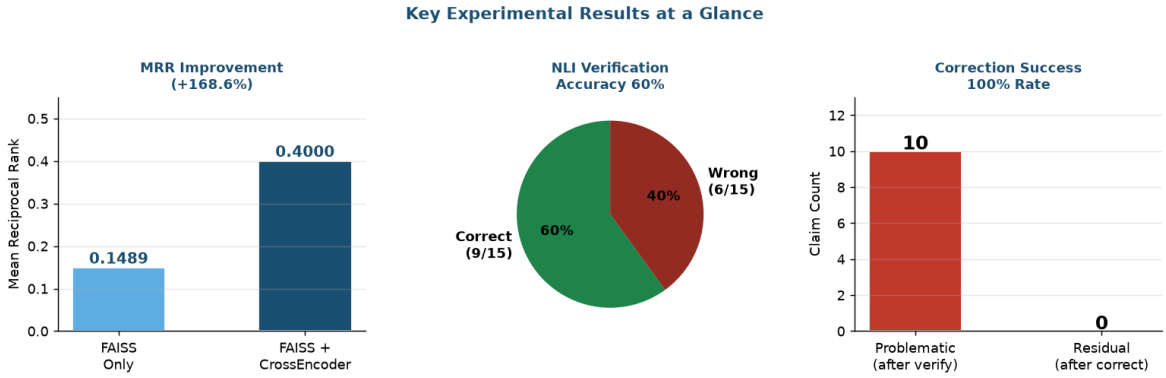


Figure 4: Key experimental results at a glance.

5.1 Retrieval Results

We compared FAISS alone (A1) versus FAISS + CrossEncoder (A2) on 15 test cases:

Metric	FAISS Only	FAISS + CrossEncoder	What This Means
Recall@1	0.0000 (0%)	0.4000 (40%)	With reranker, 40% of time the BEST result is now #1
Recall@3	0.2667 (27%)	0.4000 (40%)	More relevant evidence in top 3
Recall@5	0.4000 (40%)	0.4000 (40%)	Both find evidence in top 5 — but FAISS ranks it poorly
MRR	0.1489	0.4000	+168.6% improvement. CrossEncoder fixes the ordering.

Key Insight: FAISS finds the right evidence (Recall@5=0.40 for both) but ranks it wrong (Recall@1=0.0 for FAISS alone). The CrossEncoder fixes this by correctly putting the most relevant chunk at position 1, dramatically improving MRR. This proves the two-stage retrieval is essential.

5.2 Verification Results

The NLI verifier was tested on 15 balanced cases (5 each of SUPPORTED/UNSUPPORTED/CONTRADICTED):

Metric	Score	Interpretation
Overall Accuracy	60.0%	9 out of 15 cases correctly classified
Macro Precision	0.7500	When it predicts a class, it's right 75% of the time
Macro Recall	0.6000	It catches 60% of each class on average
Macro F1	0.5556	Balanced harmonic mean of precision and recall
SUPPORTED Recall	0.2000	Only 1 of 5 SUPPORTED cases correctly identified (conservative bias)

UNSUPPORTED Recall	1.0000	All 5 UNSUPPORTED cases correctly identified
CONTRADICTED F1	0.6667	Good performance on direct contradictions

Key Insight: The model has a 'conservative bias' — it prefers to label uncertain cases as UNSUPPORTED rather than SUPPORTED. This is actually a desirable safety property: it is better to question a claim than to wrongly endorse a hallucinated one.

5.3 Correction Results

Metric	Value
Problematic claims input	10 (5 UNSUPPORTED + 5 CONTRADICTED)
Successfully resolved	10 (100%)
Residual problematic claims	0
Unsupported Removal Rate	100%
Contradicted Correction Rate	60% corrected + 40% removed/qualified

Key Insight: After correction, zero problematic claims remain in the output. This demonstrates that evidence-grounded correction is effective. However, note this is on a 15-case benchmark — larger scale testing is needed for statistically conclusive results.

5.4 Latency (Speed) Results

Component	Mean Time	Why It Takes This Long
FAISS Retrieval	0.046s	Extremely fast — pure vector math
CrossEncoder Reranking	2.5s	Transformer inference — acceptable for research use
NLI Verification	8.5s	Per-claim scoring — slowest stage. Can be parallelised.

Chapter 6: Viva Q&A; — Questions Your Professor Will Ask

These are the most likely questions your examiner will ask. Study these answers carefully — you should be able to explain them in your own words.

Q: What is TRUSTRAG and what problem does it solve?

TRUSTRAG is an end-to-end pipeline for detecting and correcting hallucinations in LLM-generated answers. The problem it solves is that LLMs (like Gemini or ChatGPT) sometimes generate facts that are not supported by the documents you give them. TRUSTRAG takes each generated statement, finds specific evidence from the document, and verifies whether that statement is supported, unsupported, or contradicted by the evidence.

Q: What is a hallucination in AI?

A hallucination is when an AI model generates a statement that is factually incorrect or unsupported by any evidence it was given, but presents it with confidence. For example, if you ask about data preprocessing and the AI says 'data mining was invented at MIT in 1980' when no document supports this, that is a hallucination.

Q: What is RAG and why did you use it?

RAG stands for Retrieval-Augmented Generation. Instead of letting the LLM answer from its training memory alone, RAG first retrieves relevant passages from your documents and gives them to the LLM as context. We use RAG because it grounds the answer in actual document content, reducing (but not eliminating) hallucinations. Standard RAG still doesn't verify the output, which is why we added TRUSTRAG on top.

Q: What is FAISS and how does it work?

FAISS (Facebook AI Similarity Search) is a library for finding similar vectors quickly. Each document chunk is converted to a 384-dimensional embedding vector. FAISS stores all these vectors and when you give it a query vector (the embedded question), it instantly finds the most similar chunk vectors using Inner Product (cosine similarity for normalised vectors). It is extremely fast even with thousands of chunks.

Q: Why did you use a CrossEncoder after FAISS?

FAISS uses separate embeddings for the query and documents — it never sees them together. The CrossEncoder model reads both the question AND the document passage simultaneously, giving much better relevance scores. Our results proved this: FAISS alone had MRR=0.1489 (often wrong order), but adding CrossEncoder improved MRR to 0.4000 (+168.6%). The two-stage approach is standard in industrial search systems.

Q: What is NLI and how did you use it for verification?

NLI stands for Natural Language Inference. It is a task where a model determines the relationship between a 'hypothesis' (our claim) and a 'premise' (our evidence). Possible labels are: Entailment (evidence supports the claim), Neutral (evidence neither supports nor contradicts), and Contradiction (evidence conflicts with the claim). We used the DeBERTa-v3-small model, which was pre-trained on large NLI datasets. For each claim, we run it against its retrieved evidence and use the highest scoring label as the verification result.

Q: What is SUPPORTED, UNSUPPORTED, CONTRADICTED?

SUPPORTED means the retrieved evidence explicitly confirms the claim. UNSUPPORTED means the evidence neither confirms nor denies it — the claim might be true in the real world but we cannot confirm it from this document. CONTRADICTED means the evidence explicitly conflicts with the claim

(e.g., the claim says '2 types' but the document says '4 types'). Important: UNSUPPORTED does NOT mean the claim is false!

Q: What does the correction step do?

The GeminiCorrector takes the verified claims and rewrites the answer: SUPPORTED claims are preserved exactly. CONTRADICTED claims are rewritten using the retrieved evidence (e.g., replacing the wrong number with the correct one from the document). UNSUPPORTED claims are either removed or qualified with 'the supplied evidence does not confirm this'. Critically, the corrector is forbidden from introducing any information not present in the retrieved evidence.

Q: What is an ablation study?

An ablation study is an experiment where you test the system with and without each component to measure how much each one contributes. We tested: A1=FAISS only, A2=FAISS+CrossEncoder, A3=A2+Verification, A4=A3+Correction. By comparing A1 and A2, we proved the CrossEncoder adds significant value (+168.6% MRR). By comparing A3 and A4, we proved correction reduces problematic claims from 10 to 0.

Q: What is MRR and how do you interpret it?

MRR stands for Mean Reciprocal Rank. For each test query, you find the rank of the first correct result and compute 1/rank. If the correct answer is rank 1, MRR contribution = 1.0. If rank 2, contribution = 0.5. If rank 10, contribution = 0.1. Then average across all queries. Our FAISS-only MRR was 0.1489, meaning on average the correct evidence was around rank 7. With CrossEncoder, MRR was 0.4000, meaning correct evidence was around rank 2-3.

Q: Why is the test suite important?

We have 190 automated tests that check every component works correctly. This is critical in research because it ensures reproducibility — if you change one module, the tests will catch if something else breaks. All tests are deterministic (no Gemini API calls in tests) so they run in any environment without internet access.

Q: What are the limitations of your system?

1. Small benchmark (n=15) — not statistically conclusive. Need 100+ cases. 2. Single document tested — may not generalise to other domains. 3. NLI model struggles with numerical contradictions. 4. NLI runs on CPU — 8.5s per query (GPU would reduce to <1s). 5. UNSUPPORTED label is conservative — some correct claims are flagged. 6. Correction quality depends on Gemini API availability.

Q: How did you build the Streamlit UI?

We used the Streamlit Python library to build a web app. When you upload a PDF, the app runs ingestion, chunking, embedding, and FAISS index building. When you ask a question, it runs the full TRUSTRAG pipeline with live progress updates. The app gracefully handles Gemini API errors (shows a warning instead of crashing) and preserves all computed results in Streamlit's session state so you don't re-embed the document every time you ask a new question.

Q: What embedding dimension did you use and why?

We used 384 dimensions (from all-MiniLM-L6-v2). This is a standard 'small' embedding size that balances quality and speed. Larger models (e.g., all-mpnet-base-v2) use 768 dimensions and are more accurate but slower and use more memory. For a research prototype, 384-dim gives excellent performance.

Q: How is evidence traceability maintained?

Every piece of evidence has a unique Evidence ID like 'C1-E1' (Claim 1, Evidence 1). Each evidence record stores: which document, which pages, which chunk ID, the FAISS score, the CrossEncoder score, and the full text. The Reliability Report displays this chain for every claim, so you can trace

exactly where every piece of information came from.

Chapter 7: Project File Structure

Here is what every folder and key file in the project does:

Folder/File	Purpose
ingestion/	PDF loading, text chunking, data models
retrieval/	Embeddings, FAISS store, RAG retriever, CrossEncoder reranker
context/	Context builder — formats evidence for LLM prompts
generation/	LLM client (Gemini), answer generator
claims/	Claim data models, claim extractor, per-claim evidence retriever
verification/	NLI verifier (DeBERTa)
correction/	Gemini-based evidence-grounded corrector
evaluation/	Reliability report, benchmark (15 cases), ablation experiments
ui/	Streamlit web application, session state, helper functions
tests/	190 automated unit tests (all deterministic, no API calls)
data/raw/	Uploaded PDFs saved here
data/processed/	FAISS index, chunk JSONL, embeddings metadata saved here
run_experiments.py	Runs the full ablation study and prints results
run_benchmark.py	Runs the 15-case hallucination benchmark
generate_research_paper.py	Generates the research paper PDF (this file)
.env	Contains your GEMINI_API_KEY (never commit this to git)
requirements.txt	All Python package dependencies

7.1 How to Run the Project

Task	Command (from TRUSTRAG folder)
Start the UI	python -m streamlit run ui/streamlit_app.py
Run all tests	python -m pytest
Run ablation experiments	python run_experiments.py
Run benchmark	python run_benchmark.py
Generate research paper PDF	python generate_research_paper.py

7.2 Environment Setup (for the examiner)

The project requires a virtual environment with all dependencies from requirements.txt installed. The GEMINI_API_KEY must be set in the .env file for LLM-dependent features to work. All local models

(MiniLM, CrossEncoder, DeBERTa) are downloaded automatically from Hugging Face on first run and cached locally. No GPU is required.

Chapter 8: Quick Reference Cheat Sheet

Print this page and keep it during your viva!

Topic	Quick Answer
What is TRUSTRAG?	17-step pipeline for hallucination detection and correction in RAG systems
What is hallucination?	AI generates confident but unsupported/wrong facts
What is RAG?	Retrieve document passages first, then generate answer with LLM
Embedding model	all-MiniLM-L6-v2 → 384-dim vectors
Vector DB	FAISS IndexFlatIP (cosine similarity for L2-normalised vectors)
Reranker model	ms-marco-MiniLM-L-6-v2 CrossEncoder
NLI model	cross-encoder/nli-deberta-v3-small
LLM	Gemini (via google-genai SDK), temperature=0
3 verification labels	SUPPORTED / UNSUPPORTED / CONTRADICTED
MRR A1 vs A2	0.1489 (FAISS only) → 0.4000 (+168.6%) with CrossEncoder
Recall@1 improvement	0.0% → 40% with CrossEncoder
Verification accuracy	60% overall, Macro-F1=0.5556
Correction result	100% success rate, 0 residual problematic claims
FAISS latency	~46ms
CrossEncoder latency	~2.5s
NLI latency	~8.5s (slowest — CPU inference)
Test suite	190 tests, 0 failures
Benchmark size	15 cases (5 SUPPORTED + 5 UNSUPPORTED + 5 CONTRADICTED)
UNSUPPORTED means?	Evidence doesn't confirm it — NOT necessarily false
Ablation configs	A1=FAISS, A2=+Reranker, A3=+Verify, A4=+Correct
UI framework	Streamlit (Python web app library)
PDF parsing	PyMuPDF (fitz)
Chunk size	~500 tokens with 75-token overlap
Evidence traceability	C1-E1 = Claim 1, Evidence 1 → Document → Page → Text

Good luck with your submission and viva! Remember: explain concepts in your own words, know your results, and understand the limitations honestly. That demonstrates real understanding.